

Лабораторная работа №17

Реализация мини-проекта с использованием алгоритмов и структур данных

Цель работы:

1. Завершить разработку проекта, начатого в ЛР23
2. Реализовать методы, использующие изученные алгоритмы и структуры данных
3. Освоить сериализацию (сохранение и загрузку данных в файл)
4. Освоить механизм обработки исключительных ситуаций (try-catch-throw)
5. Создать пользовательский интерфейс (меню) для взаимодействия с системой
6. Защитить разработанный проект

Теоретическое обоснование

1. Сериализация объектов

Сериализация - это процесс преобразования состояния объекта в форму, пригодную для сохранения (в файл) или передачи (по сети). Обратный процесс называется десериализацией.

Зачем нужна сериализация?

Причина	Описание
Сохранение данных	Данные программы не теряются после завершения работы
Передача по сети	Объекты можно отправлять между программами
Межпрограммный обмен	Разные программы могут обмениваться данными через файлы

Способы сериализации в C++

Способ	Описание	Сложность реализации	Читаемость файла
Текстовый	Данные записываются в текстовом виде	Средняя	Высокая

Способ	Описание	Сложность реализации	Читаемость файла
	(CSV, JSON, XML)		
Бинарный	Данные записываются в бинарном виде (как в памяти)	Низкая	Нет
С использованием библиотек	Boost.Serialization, Protobuf, nlohmann/json	Высокая	Зависит от библиотеки

Пример текстовой сериализации

```
#include <fstream>
```

```
#include <string>
```

```
#include <vector>
```

```
class Book {
```

```
private:
```

```
    std::string title;
```

```
    std::string author;
```

```
    int year;
```

```
public:
```

```
    // Конструкторы
```

```
    Book() : title(""), author(""), year(0) {}
```

```
    Book(const std::string& t, const std::string& a, int y)
        : title(t), author(a), year(y) {}
```

```
    // Геттеры
```

```
    std::string getTitle() const { return title; }
```

```
    std::string getAuthor() const { return author; }
```

```
    int getYear() const { return year; }
```

```
    // Сериализация (сохранение)
```

```
    void saveToFile(std::ofstream& out) const {
```

```
        out << title << std::endl;
```

```
        out << author << std::endl;
```

```

        out << year << std::endl;
    }

    // Десериализация (загрузка)
    void loadFromFile(std::ifstream& in) {
        std::getline(in, title);
        std::getline(in, author);
        in >> year;
        in.ignore(); // пропускаем символ новой строки
    }
};

```

Пример бинарной сериализации

```

class BookBinary {
private:
    char title[100];
    char author[50];
    int year;

public:
    void saveToFile(std::ofstream& out) const {
        out.write((const char*)this, sizeof BookBinary);
    }

    void loadFromFile(std::ifstream& in) {
        in.read((char*)this, sizeof BookBinary);
    }
};

```

2. Обработка исключительных ситуаций

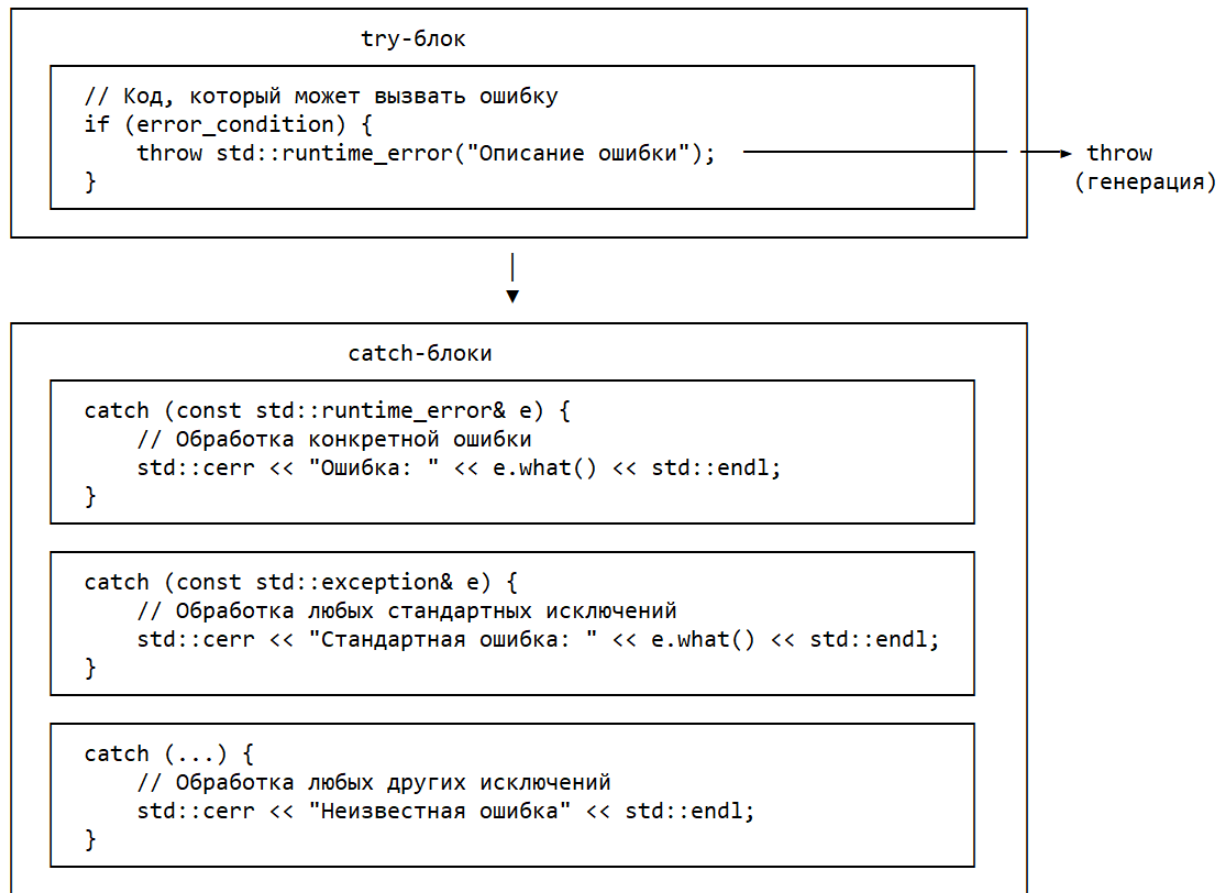
Исключение (exception) - это событие, которое нарушает нормальный ход выполнения программы. Механизм исключений позволяет отделить код обработки ошибок от основного кода программы.

Основные конструкции

Конструкция	Назначение
try	Блок кода, в котором могут возникнуть исключения
throw	Генерация исключения (передача управления обработчику)

Конструкция	Назначение
catch	Блок обработки исключения определённого типа

Схема работы механизма исключений



Стандартные классы исключений

Класс исключения	Назначение	Заголовочный файл
<code>std::exception</code>	Базовый класс для всех исключений	<code><exception></code>
<code>std::runtime_error</code>	Ошибки времени выполнения	<code><stdexcept></code>
<code>std::logic_error</code>	Логические ошибки в программе	<code><stdexcept></code>
<code>std::out_of_range</code>	Выход за границы диапазона	<code><stdexcept></code>

Класс исключения	Назначение	Заголовочный файл
<code>std::invalid_argument</code>	Некорректный аргумент функции	<code><stdexcept></code>
<code>std::bad_alloc</code>	Ошибка выделения памяти	<code><new></code>

Примеры использования исключений

Пример 1: Проверка индекса

```
int getElement(const std::vector<int>& vec, int index) {
    if (index < 0 || index >= static_cast<int>(vec.size())) {
        throw std::out_of_range("Индекс " + std::to_string(index) +
            " вне диапазона (0-" +
            std::to_string(vec.size() - 1) + ")");
    }
    return vec[index];
}
```

// Использование

```
try {
    int value = getElement(myVector, 10);
    std::cout << "Значение: " << value << std::endl;
}
catch (const std::out_of_range& e) {
    std::cerr << "Ошибка: " << e.what() << std::endl;
}
```

Пример 2: Работа с файлами

```
void loadFromFile(const std::string& filename) {
    std::ifstream file(filename);
    if (!file.is_open()) {
        throw std::runtime_error("Не удалось открыть файл " +
filename);
    }

    // Чтение данных...

    if (bad_format) {
        throw std::runtime_error("Некорректный формат данных в
файле");
    }
}
```

```

    }
}

// Использование
try {
    loadFromFile("data.txt");
    std::cout << "Данные загружены" << std::endl;
}
catch (const std::runtime_error& e) {
    std::cerr << "Ошибка загрузки: " << e.what() << std::endl;
    // Программа продолжает работу с пустой коллекцией
}

```

Пример 3: Пользовательский ввод

```

int getPositiveNumber() {
    int value;
    std::cin >> value;

    if (std::cin.fail()) {
        std::cin.clear();
        std::cin.ignore(32767, '\n');
        throw std::invalid_argument("Введено не число");
    }

    if (value <= 0) {
        throw std::invalid_argument("Число должно быть
положительным");
    }

    return value;
}

// Использование
bool valid = false;
while (!valid) {
    try {
        std::cout << "Введите положительное число: ";
        int num = getPositiveNumber();
        std::cout << "Вы ввели: " << num << std::endl;
        valid = true;
    }
    catch (const std::invalid_argument& e) {

```

```

        std::cerr << "Ошибка: " << e.what() << ". Попробуйте снова." <<
std::endl;
    }
}

```

Создание собственных классов исключений

```

#include <stdexcept>
#include <string>

// Собственный класс исключения для ошибок в библиотеке
class LibraryException : public std::runtime_error {
public:
    LibraryException(const std::string& msg) : std::runtime_error(msg)
{}

};

// Специализированные исключения
class FileNotFoundException : public LibraryException {
public:
    FileNotFoundException(const std::string& filename)
        : LibraryException("Файл не найден: " + filename) {}
};

class InvalidDataFormatException : public LibraryException {
public:
    InvalidDataFormatException(const std::string& details)
        : LibraryException("Неверный формат данных: " + details) {}
};

// Использование
try {
    throw FileNotFoundException("data.txt");
}
catch (const LibraryException& e) {
    std::cerr << "Ошибка библиотеки: " << e.what() << std::endl;
}
catch (const std::exception& e) {
    std::cerr << "Общая ошибка: " << e.what() << std::endl;
}

```

Важные правила работы с исключениями

Правило	Объяснение
Деструкторы не должны выбрасывать исключения	Исключение в деструкторе может вызвать <code>std::terminate()</code>
Перехватывайте исключения по ссылке	<code>catch (const std::exception& e)</code> - избегает срезки объекта
Не используйте исключения для нормального потока управления	Исключения - для исключительных ситуаций
Всегда обрабатывайте возможные исключения	Неперехваченное исключение вызывает аварийное завершение

Требования к проекту

Функциональные требования

№	Требование	Описание
1	Иерархия классов	Минимум базовый абстрактный класс + 2 производных
2	Виртуальные методы	Полиморфное поведение через указатель на базовый класс
3	Контейнер STL	Использование <code>std::vector</code> (или <code>std::list</code> , <code>std::map</code>)
4	Сортировка	Сортировка коллекции по заданному критерию
5	Поиск	Поиск элементов по различным критериям
6	Сериализация	Сохранение и загрузка данных в/из файла (текстового или бинарного)
7	Обработка исключений	Обработка ошибок с помощью <code>try-catch-throw</code>

№	Требование	Описание
8	Пользовательский интерфейс	Консольное меню с выбором действий

Обязательные ситуации для использования исключений

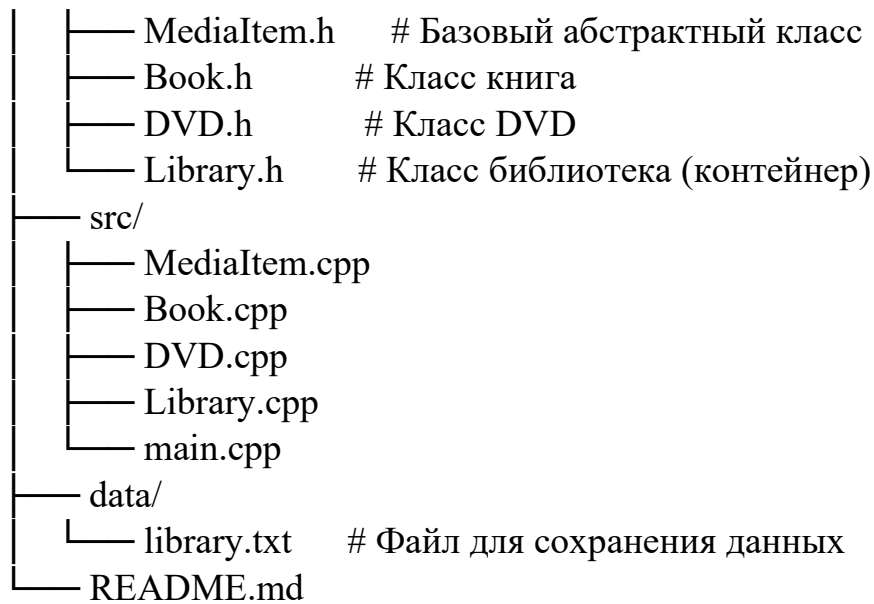
№	Ситуация	Тип исключения	Обработка
1	Ошибка открытия файла при загрузке	<code>std::runtime_error</code>	Сообщение, продолжение работы
2	Ошибка записи в файл	<code>std::runtime_error</code>	Сообщение, предложение другого имени
3	Некорректный ввод данных (не число)	<code>std::invalid_argument</code>	Повтор запроса
4	Некорректный ввод данных (диапазон)	<code>std::out_of_range</code>	Повтор запроса
5	Попытка удалить несуществующий элемент	<code>std::out_of_range</code>	Сообщение пользователю
6	Попытка доступа к пустой коллекции	<code>std::logic_error</code>	Сообщение пользователю

Пример выполнения практической части

Сквозной пример: Система управления библиотекой

Структура проекта

```
library_project/
└── include/
```



Файл: MediaItem.h

```
#ifndef MEDIA_ITEM_H
#define MEDIA_ITEM_H
```

```
#include <string>
#include <iostream>
#include <fstream>
```

```
// Базовый абстрактный класс
```

```
class MediaItem {
```

```
protected:
```

```
    std::string title;
```

```
    int year;
```

```
    std::string genre;
```

```
public:
```

```
    // Конструкторы
```

```
    MediaItem();
```

```
    MediaItem(const std::string& title, int year, const std::string& genre);
```

```
    virtual ~MediaItem() {}
```

```
    // Геттеры (не виртуальные)
```

```
    std::string getTitle() const;
```

```
    int getYear() const;
```

```
    std::string getGenre() const;
```

```
    // Сеттеры (с проверкой)
```

```
    void setTitle(const std::string& title);
```

```

void setYear(int year);
void setGenre(const std::string& genre);

// Чисто виртуальные методы
virtual double getDuration() const = 0;    // длительность (мин)
virtual std::string getType() const = 0;    // тип объекта
virtual void printInfo() const = 0;        // вывод информации

// Виртуальный метод с реализацией по умолчанию
virtual bool isLong() const {
    return getDuration() > 120;
}

// Сериализация
virtual void saveToFile(std::ofstream& out) const = 0;
virtual void loadFromFile(std::ifstream& in) = 0;
};

#endif
Файл: MediaItem.cpp

#include "MediaItem.h"
#include <stdexcept>

MediaItem::MediaItem() : title(""), year(0), genre("") {}

MediaItem::MediaItem(const std::string& t, int y, const std::string& g)
    : title(t), year(y), genre(g) {}

std::string MediaItem::getTitle() const { return title; }
int MediaItem::getYear() const { return year; }
std::string MediaItem::getGenre() const { return genre; }

void MediaItem::setTitle(const std::string& t) {
    if (t.empty()) {
        throw std::invalid_argument("Название не может быть пустым");
    }
    title = t;
}

void MediaItem::setYear(int y) {
    if (y < 0 || y > 2025) {

```

```

        throw std::out_of_range("Год должен быть от 0 до 2025");
    }
    year = y;
}

void MediaItem::setGenre(const std::string& g) {
    genre = g;
}

```

Файл: Book.h

```

#ifndef BOOK_H
#define BOOK_H

#include "MediaItem.h"

class Book : public MediaItem {
private:
    int pages;
    std::string author;

public:
    Book();
    Book(const std::string& title, int year, const std::string& genre,
        int pages, const std::string& author);

    // Геттеры
    int getPages() const;
    std::string getAuthor() const;

    // Переопределение виртуальных методов
    double getDuration() const override;
    std::string getType() const override;
    void printInfo() const override;

    // Сериализация
    void saveToFile(std::ofstream& out) const override;
    void loadFromFile(std::ifstream& in) override;

    // Специфичный метод
    void read() const;
};

```

```
#endif
```

Файл: Book.cpp

```
#include "Book.h"
```

```
#include <stdexcept>
```

```
Book::Book() : MediaItem(), pages(0), author("") {}
```

```
Book::Book(const std::string& title, int year, const std::string& genre,  
            int pages, const std::string& author)  
    : MediaItem(title, year, genre), pages(pages), author(author) {}
```

```
int Book::getPages() const { return pages; }
```

```
std::string Book::getAuthor() const { return author; }
```

```
double Book::getDuration() const {
```

```
    // Для книг "длительность" - время чтения (страницы / 2)
```

```
    return pages / 2.0;
```

```
}
```

```
std::string Book::getType() const {
```

```
    return "Книга";
```

```
}
```

```
void Book::printInfo() const {
```

```
    std::cout << "=== Книга ===" << std::endl;
```

```
    std::cout << "Название: " << title << std::endl;
```

```
    std::cout << "Автор: " << author << std::endl;
```

```
    std::cout << "Год: " << year << std::endl;
```

```
    std::cout << "Жанр: " << genre << std::endl;
```

```
    std::cout << "Страниц: " << pages << std::endl;
```

```
    std::cout << "Время чтения: " << getDuration() << " мин" <<
```

```
std::endl;
```

```
}
```

```
void Book::saveToFile(std::ofstream& out) const {
```

```
    out << "Book" << std::endl;
```

```
    out << title << std::endl;
```

```
    out << year << std::endl;
```

```
    out << genre << std::endl;
```

```
    out << pages << std::endl;
```

```
    out << author << std::endl;
```

```

    }

    void Book::loadFromFile(std::ifstream& in) {
        std::getline(in, title);
        in >> year;
        in.ignore();
        std::getline(in, genre);
        in >> pages;
        in.ignore();
        std::getline(in, author);
    }

    void Book::read() const {
        std::cout << "Читаем книгу \"" << title << "\" (" << pages << "
страниц)" << std::endl;
    }
}
Файл: DVD.h
cpp
#ifndef DVD_H
#define DVD_H

#include "MediaItem.h"

class DVD : public MediaItem {
private:
    int duration    // длительность в минутах
    std::string director;

public:
    DVD();
    DVD(const std::string& title, int year, const std::string& genre,
        int duration, const std::string& director);

    // Геттеры
    int getDurationMinutes() const;
    std::string getDirector() const;

    // Переопределение виртуальных методов
    double getDuration() const override;
    std::string getType() const override;
    void printInfo() const override;

```

```

// Сериализация
void saveToFile(std::ofstream& out) const override;
void loadFromFile(std::ifstream& in) override;

// Специфичный метод
void watch() const;
};

#endif

```

Файл: DVD.cpp

```

#include "DVD.h"
#include <stdexcept>

DVD::DVD() : MediaItem(), duration(0), director("") {}

DVD::DVD(const std::string& title, int year, const std::string& genre,
          int duration, const std::string& director)
    : MediaItem(title, year, genre), duration(duration), director(director)
{}

int DVD::getDurationMinutes() const { return duration; }
std::string DVD::getDirector() const { return director; }

double DVD::getDuration() const {
    return duration;
}

std::string DVD::getType() const {
    return "DVD";
}

void DVD::printInfo() const {
    std::cout << "=== DVD ===" << std::endl;
    std::cout << "Название: " << title << std::endl;
    std::cout << "Режиссёр: " << director << std::endl;
    std::cout << "Год: " << year << std::endl;
    std::cout << "Жанр: " << genre << std::endl;
    std::cout << "Длительность: " << duration << " мин" << std::endl;
}

void DVD::saveToFile(std::ofstream& out) const {

```

```

        out << "DVD" << std::endl;
        out << title << std::endl;
        out << year << std::endl;
        out << genre << std::endl;
        out << duration << std::endl;
        out << director << std::endl;
    }

    void DVD::loadFromFile(std::ifstream& in) {
        std::getline(in, title);
        in >> year;
        in.ignore();
        std::getline(in, genre);
        in >> duration;
        in.ignore();
        std::getline(in, director);
    }

    void DVD::watch() const {
        std::cout << "Смотрим DVD \"" << title << "\" (" << duration << "
минут)" << std::endl;
    }

```

Файл: Library.h

```

#ifndef LIBRARY_H
#define LIBRARY_H

#include <vector>
#include <memory>
#include <string>
#include "MediaItem.h"

class Library {
private:
    std::vector<std::unique_ptr<MediaItem>> items;

public:
    // Добавление элементов
    void addBook();
    void addDVD();

```

```

// Удаление элементов
void removeItem(int index);
void removeByTitle(const std::string& title);

// Просмотр
void showAll() const;
void showLongItems() const; // элементы длительностью > 120
МИН

// Сортировка
void sortByYear();
void sortByTitle();
void sortByDuration();

// Поиск
void findByTitle(const std::string& title) const;
void findByYearRange(int minYear, int maxYear) const;
void findByGenre(const std::string& genre) const;

// Сериализация
void saveToFile(const std::string& filename) const;
void loadFromFile(const std::string& filename);

// Служебные методы
int getSize() const;
bool isEmpty() const;
void clear();
};

#endif

```

Файл: Library.cpp

```

#include "Library.h"
#include "Book.h"
#include "DVD.h"
#include <iostream>
#include <algorithm>
#include <fstream>
#include <stdexcept>
#include <memory>

```

```

// -----
// Добавление элементов
// -----

void Library::addBook() {
    std::string title, genre, author;
    int year, pages;

    try {
        std::cout << "Введите название книги: ";
        std::cin.ignore();
        std::getline(std::cin, title);

        std::cout << "Введите год издания: ";
        std::cin >> year;
        if (std::cin.fail()) {
            std::cin.clear();
            std::cin.ignore(32767, '\n');
            throw std::invalid_argument("Год должен быть числом");
        }

        std::cout << "Введите жанр: ";
        std::cin.ignore();
        std::getline(std::cin, genre);

        std::cout << "Введите количество страниц: ";
        std::cin >> pages;
        if (std::cin.fail() || pages <= 0) {
            std::cin.clear();
            std::cin.ignore(32767, '\n');
            throw std::invalid_argument("Количество страниц должно
быть положительным числом");
        }

        std::cout << "Введите автора: ";
        std::cin.ignore();
        std::getline(std::cin, author);

        items.push_back(std::make_unique<Book>(title, year, genre,
pages, author));
        std::cout << "Книга добавлена!" << std::endl;
    }
}

```

```

        catch (const std::exception& e) {
            std::cerr << "Ошибка при добавлении книги: " << e.what() <<
std::endl;
        }
    }

void Library::addDVD() {
    std::string title, genre, director,
    int year, duration;

    try {
        std::cout << "Введите название DVD: ";
        std::cin.ignore();
        std::getline(std::cin, title);

        std::cout << "Введите год выпуска: ";
        std::cin >> year;
        if (std::cin.fail()) {
            std::cin.clear();
            std::cin.ignore(32767, '\n');
            throw std::invalid_argument("Год должен быть числом");
        }

        std::cout << "Введите жанр: ";
        std::cin.ignore();
        std::getline(std::cin, genre);

        std::cout << "Введите длительность (минуты): ";
        std::cin >> duration;
        if (std::cin.fail() || duration <= 0) {
            std::cin.clear();
            std::cin.ignore(32767, '\n');
            throw std::invalid_argument("Длительность должна быть
положительным числом");
        }

        std::cout << "Введите режиссёра: ";
        std::cin.ignore();
        std::getline(std::cin, director);

        items.push_back(std::make_unique<DVD>(title, year, genre,
duration, director));
    }
}

```

```

        std::cout << "DVD добавлен!" << std::endl;
    }
    catch (const std::exception& e) {
        std::cerr << "Ошибка при добавлении DVD: " << e.what() <<
std::endl;
    }
}

// -----
// Удаление элементов
// -----

void Library::removeItem(int index) {
    try {
        if (index < 0 || index >= static_cast<int>(items.size())) {
            throw std::out_of_range("Индекс " + std::to_string(index) +
                " вне диапазона (0-" +
                std::to_string(items.size() - 1) + ")");
        }
        items.erase(items.begin() + index);
        std::cout << "Элемент удалён" << std::endl;
    }
    catch (const std::out_of_range& e) {
        std::cerr << "Ошибка удаления: " << e.what() << std::endl;
    }
}

void Library::removeByTitle(const std::string& title) {
    auto it = std::find_if(items.begin(), items.end(),
        [&title](const std::unique_ptr<MediaItem>& item) {
            return item->getTitle() == title;
        });

    if (it != items.end()) {
        items.erase(it);
        std::cout << "Элемент \"" << title << "\" удалён" << std::endl;
    } else {
        std::cout << "Элемент \"" << title << "\" не найден" << std::endl;
    }
}

// -----

```

```

// Просмотр
// -----

void Library::showAll() const {
    if (items.empty()) {
        std::cout << "Библиотека пуста" << std::endl;
        return;
    }

    std::cout << "\n=== Содержимое библиотеки ===" << std::endl;
    for (size_t i = 0; i < items.size(); ++i) {
        std::cout << i + 1 << ". ";
        items[i]->printInfo();
        std::cout << std::endl;
    }
}

void Library::showLongItems() const {
    if (items.empty()) {
        std::cout << "Библиотека пуста" << std::endl;
        return;
    }

    std::cout << "\n=== Длинные произведения (> 120 мин) ===" <<
std::endl;
    bool found = false;
    for (const auto& item : items) {
        if (item->isLong()) {
            item->printInfo();
            std::cout << std::endl;
            found = true;
        }
    }

    if (!found) {
        std::cout << "Нет произведений длительностью более 120
минут" << std::endl;
    }
}

// -----
// Сортировка
// -----

```

```

void Library::sortByYear() {
    std::sort(items.begin(), items.end(),
        [](const std::unique_ptr<MediaItem>& a, const
std::unique_ptr<MediaItem>& b) {
            return a->getYear() < b->getYear();
        });
    std::cout << "Коллекция отсортирована по году" << std::endl;
    showAll();
}

```

```

void Library::sortByTitle() {
    std::sort(items.begin(), items.end(),
        [](const std::unique_ptr<MediaItem>& a, const
std::unique_ptr<MediaItem>& b) {
            return a->getTitle() < b->getTitle();
        });
    std::cout << "Коллекция отсортирована по названию" <<
std::endl;
    showAll();
}

```

```

void Library::sortByDuration() {
    std::sort(items.begin(), items.end(),
        [](const std::unique_ptr<MediaItem>& a, const
std::unique_ptr<MediaItem>& b) {
            return a->getDuration() < b->getDuration();
        });
    std::cout << "Коллекция отсортирована по длительности" <<
std::endl;
    showAll();
}

```

```

// -----
// Поиск
// -----

```

```

void Library::findByTitle(const std::string& title) const {
    auto it = std::find_if(items.begin(), items.end(),
        [&title](const std::unique_ptr<MediaItem>& item) {
            return item->getTitle() == title;
        });
}

```

```

        if (it != items.end()) {
            std::cout << "Найдено:" << std::endl;
            (*it)->printInfo();
        } else {
            std::cout << "Элемент \"" << title << "\" не найден" << std::endl;
        }
    }

void Library::findByYearRange(int minYear int maxYear) const {
    try {
        if (minYear > maxYear) {
            throw std::invalid_argument("Минимальный год не может
быть больше максимального");
        }

        bool found = false;
        for (const auto& item : items) {
            int year = item->getYear();
            if (year >= minYear && year <= maxYear) {
                item->printInfo();
                std::cout << std::endl;
                found = true;
            }
        }

        if (!found) {
            std::cout << "Нет элементов в диапазоне годов " << minYear
<< "-" << maxYear << std::endl;
        }
    }
    catch (const std::invalid_argument& e) {
        std::cerr << "Ошибка поиска: " << e.what() << std::endl;
    }
}

void Library::findByGenre(const std::string& genre) const {
    bool found = false;
    for (const auto& item : items) {
        if (item->getGenre() == genre) {
            item->printInfo();
            std::cout << std::endl;
        }
    }
}

```

```

        found = true;
    }
}

if (!found) {
    std::cout << "Нет элементов в жанре \"" << genre << "\"" <<
std::endl;
}

// -----
// Сериализация
// -----

void Library::saveToFile(const std::string& filename) const {
    try {
        std::ofstream out(filename);
        if (!out.is_open()) {
            throw std::runtime_error("Не удалось открыть файл " +
filename + " для записи");
        }

        out << items.size() << std::endl;

        for (const auto& item : items) {
            item->saveToFile(out);
        }

        std::cout << "Данные сохранены в файл " << filename <<
std::endl;
    }
    catch (const std::runtime_error& e) {
        std::cerr << "Ошибка сохранения: " << e.what() << std::endl;
    }
}

void Library::loadFromFile(const std::string& filename) {
    try {
        std::ifstream in(filename);
        if (!in.is_open()) {
            throw std::runtime_error("Не удалось открыть файл " +
filename + " для чтения");
        }
    }
}

```

```

    }

    clear();

    int size;
    in >> size;
    in ignore();

    for (int i = 0; i < size; ++i) {
        std::string type;
        std::getline(in, type);

        if (type == "Book") {
            auto book = std::make_unique<Book>();
            book->loadFromFile(in);
            items.push_back(std::move(book));
        } else if (type == "DVD") {
            auto dvd = std::make_unique<DVD>();
            dvd->loadFromFile(in);
            items.push_back(std::move(dvd));
        } else {
            throw std::runtime_error("Неизвестный тип объекта: " +
type);
        }
    }

    std::cout << "Данные загружены из файла " << filename <<
std::endl
    std::cout << "Загружено элементов: " << items.size() << std::endl
    }
    catch (const std::runtime_error& e) {
        std::cerr << "Ошибка загрузки: " << e.what() << std::endl;
        std::cerr << "Программа продолжит работу с пустой
коллекцией" << std::endl;
    }
}

// -----
// Служебные методы
// -----

int Library::getSize() const {

```

```

        return items.size();
    }

    bool Library::isEmpty() const {
        return items.empty();
    }

    void Library::clear() {
        items.clear();
    }

```

Файл: main.cpp

```

#include <iostream>
#include <limits>
#include "Library.h"

using namespace std;

void showMenu() {
    cout << "\n=====
<< endl
    cout << " СИСТЕМА УПРАВЛЕНИЯ БИБЛИОТЕКОЙ" << endl
    cout << "=====
<< endl
    cout << "1. Добавить книгу" << endl
    cout << "2. Добавить DVD" << endl
    cout << "3. Показать все" << endl
    cout << "4. Показать длинные произведения (>120 мин)" << endl
    cout << "5. Удалить элемент по индексу" << endl
    cout << "6. Удалить элемент по названию" << endl
    cout << "7. Сортировать по году" << endl
    cout << "8. Сортировать по названию" << endl
    cout << "9. Сортировать по длительности" << endl
    cout << "10. Найти по названию" << endl
    cout << "11. Найти по диапазону годов" << endl
    cout << "12. Найти по жанру" << endl
    cout << "13. Сохранить в файл" << endl
    cout << "14. Загрузить из файла" << endl
    cout << "15. Очистить библиотеку" << endl
    cout << "0. Выход" << endl
    cout << "-----" << endl
    cout << "Выберите действие: ";

```

```

    }

    int getChoice() {
        int choice;
        cin >> choice;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return -1;
        }

        return choice;
    }

    int getIndex(int maxSize) {
        int index;
        cout << "Введите индекс (0-" << maxSize - 1 << "): ";
        cin >> index;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            throw invalid_argument("Индекс должен быть числом");
        }

        return index;
    }

    int main() {
        setlocale(LC_ALL, "ru");

        Library lib;
        int choice;

        cout << "Добро пожаловать в систему управления библиотекой!"
<< endl;

        do {
            showMenu();
            choice = getChoice();

```

```

switch (choice) {
    case 1:
        lib.addBook();
        break;

    case 2:
        lib.addDVD();
        break;

    case 3:
        lib.showAll();
        break;

    case 4:
        lib.showLongItems();
        break;

    case 5:
        if (lib.isEmpty()) {
            cout << "Библиотека пуста, нечего удалять" << endl;
        } else {
            try {
                int index = getIndex(lib.getSize());
                lib.removeItem(index);
            }
            catch (const exception& e) {
                cerr << "Ошибка: " << e.what() << endl;
            }
        }
        break;

    case 6: {
        if (lib.isEmpty()) {
            cout << "Библиотека пуста, нечего удалять" << endl;
        } else {
            string title;
            cout << "Введите название: ";
            cin.ignore();
            getline(cin, title);
            lib.removeByTitle(title);
        }
        break;
    }
}

```

```
}
```

```
case 7:
```

```
    if (lib.isEmpty()) {  
        cout << "Библиотека пуста, нечего сортировать" << endl;  
    } else {  
        lib.sortByYear();  
    }  
    break;
```

```
case 8:
```

```
    if (lib.isEmpty()) {  
        cout << "Библиотека пуста, нечего сортировать" << endl;  
    } else {  
        lib.sortByTitle();  
    }  
    break;
```

```
case 9:
```

```
    if (lib.isEmpty()) {  
        cout << "Библиотека пуста, нечего сортировать" << endl;  
    } else {  
        lib.sortByDuration();  
    }  
    break;
```

```
case 10: {
```

```
    string title;  
    cout << "Введите название: ";  
    cin.ignore();  
    getline(cin, title);  
    lib.findByTitle(title);  
    break;  
}
```

```
case 11: {
```

```
    int minYear, maxYear;  
    cout << "Введите минимальный год: ";  
    cin >> minYear;  
    cout << "Введите максимальный год: ";  
    cin >> maxYear;  
    lib.findByYearRange(minYear, maxYear);
```

```

        break;
    }

    case 12: {
        string genre;
        cout << "Введите жанр: ";
        cin.ignore();
        getline(cin, genre);
        lib.findByGenre(genre);
        break;
    }

    case 13:
        lib.saveToFile("library.txt");
        break;

    case 14:
        lib.loadFromFile("library.txt");
        break;

    case 15:
        lib.clear();
        cout << "Библиотека очищена" << endl;
        break;

    case 0:
        cout << "До свидания!" << endl;
        break;

    default:
        cout << "Неверный выбор! Попробуйте снова." << endl;
        break;
    }
} while (choice != 0);

return 0;
}

```

Индивидуальные задания
Продвинутый уровень (15 вариантов) - дополнение

Общее задание: Разработать полноценную мини-систему с использованием наследования, полиморфизма, алгоритмов STL, сериализации и обработки исключений.

Обязательные требования:

№	Требование	Описание
1	Иерархия классов	Базовый абстрактный класс + минимум 2 производных
2	Виртуальные методы	Переопределение методов в производных классах
3	Контейнер STL	Хранение объектов в <code>std::vector</code> или <code>std::map</code>
4	Сортировка	Сортировка коллекции по разным критериям
5	Поиск	Поиск по разным критериям
6	Сериализация	Сохранение/загрузка в текстовый файл
7	Исключения	Обработка ошибок с помощью <code>try-catch-throw</code>
8	Меню	Консольное меню с выбором действий

№ вар.	Тема	Типы объектов	Критерии сортировки	Критерии поиска
1	Библиотека	Книга, DVD, Журнал	год, название, длительность	название, жанр, год

№ вар.	Тема	Типы объектов	Критерии сортировки	Критерии поиска
2	Сотрудники	Менеджер, Разработчик, Стажёр	зарплата, стаж, имя	имя, отдел, опыт
3	Транспорт	Автомобиль, Грузовик, Мотоцикл	цена, год, мощность	марка, тип, цена
4	Животные	Собака, Кошка, Птица	возраст, вес, имя	имя, вид, возраст
5	Фигуры	Круг, Прямоугольник, Треугольник	площадь, периметр	тип, площадь
6	Банковские счета	Сберегательный, Расчётный, Кредитный	баланс, процент, номер	номер, владелец
7	Студенты	Бакалавр, Магистр, Аспирант	средний балл, курс, имя	имя, курс, факультет
8	Игровые персонажи	Воин, Маг, Лучник	уровень, здоровье, сила	имя, класс, уровень
9	Электроника	Ноутбук, Смартфон, Планшет	цена, производительность	бренд, цена, модель
10	Музыка	Песня, Альбом, Исполнитель	длительность, год, рейтинг	название, исполнитель
11	Спорт	Футбол, Баскетбол, Теннис	количество игроков, популярность	название, вид
12	Еда	Фрукт, Овощ, Мясо	калории, цена, вес	название, тип

№ вар.	Тема	Типы объектов	Критерии сортировки	Критерии поиска
13	Фильмы	Боевик, Комедия, Драма	рейтинг, год, длительность	название, режиссёр
14	Квартиры	Однокомнатная, Двухкомнатная, Трёхкомнатная	цена, площадь, этаж	адрес, цена, комнат
15	Задачи	Простая, Срочная, Долгосрочная	приоритет, срок, сложность	название, исполнитель

Контрольные вопросы

1. Что такое сериализация? Для чего она нужна?
2. Какие способы сериализации в C++ вы знаете?
3. В чём преимущества и недостатки текстовой сериализации?
4. В чём преимущества и недостатки бинарной сериализации?
5. Что такое исключение? Для чего нужен механизм исключений?
6. Какие конструкции используются для работы с исключениями (try, catch, throw)?
7. В чём разница между catch (const std::exception& e) и catch (...)?
8. Какие стандартные классы исключений вы знаете?
9. Как создать собственный класс исключения?
10. Почему деструкторы не должны выбрасывать исключения?
11. Как обрабатывать ошибки ввода данных пользователем с помощью исключений?
12. Как обрабатывать ошибки при работе с файлами с помощью исключений?
13. Что такое RAII? Как он связан с исключениями?
14. Как обрабатывать ошибки выделения памяти (std::bad_alloc)?
15. В каких ситуациях в вашем проекте используются исключения?

Требования к отчёту

Титульный лист
Цель работы

Теоретическая часть (сериализация, исключения)
 Постановка задачи (согласно варианту)
 Диаграмма классов UML
 Листинг программы с комментариями
 Результаты работы (скриншоты вывода)
 Анализ использования исключений (в каких ситуациях и какие
 исключения используются)
 Ответы на контрольные вопросы
 Выводы
 Ссылка на Git-репозиторий

Критерии оценивания

Максимальный балл: 100

№	Критерий	Баллы
1	Иерархия классов (из ЛР23)	20
1.1	Базовый абстрактный класс с чисто виртуальными методами	10
1.2	Производные классы (минимум 2), переопределение методов	10
2	Реализация функциональности	25
2.1	Использование контейнеров STL (vector, list, map)	10
2.2	Реализация сортировки элементов коллекции	8
2.3	Реализация поиска элементов по критериям	7
3	Сериализация	15
3.1	Сохранение данных в файл	8
3.2	Загрузка данных из файла	7
4	Обработка исключений	15

№	Критерий	Баллы
4.1	Обработка ошибок файлового ввода/вывода	5
4.2	Обработка ошибок ввода данных пользователем	5
4.3	Обработка ошибок доступа к коллекции (выход за границы, удаление из пустого)	5
5	Пользовательский интерфейс	10
5.1	Меню с выбором действий	4
5.2	Обработка ошибок ввода (нечисловые символы и т.д.)	3
5.3	Информативные сообщения об ошибках	3
6	Защита проекта	15
6.1	Презентация проекта (5-7 минут)	4
6.2	Демонстрация работы программы	4
6.3	Ответы на вопросы по проекту	4
6.4	Качество кода и комментарии	3
Итого		100

Штрафные баллы

Нарушение	Штраф
Отсутствие виртуального деструктора в базовом классе	-15
Утечка памяти (не освобождена динамическая память)	-15

Нарушение	Штраф
Нет обработки ошибок при работе с файлами (исключениями или проверками)	-10
Нет меню или оно неработоспособно	-10
Исключения не используются там, где они необходимы (см. список обязательных ситуаций)	-10
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60